# Defensive Data Modeling in the AI Era: Facts vs. AI Opinions

*If your database schema doesn't differentiate between hard facts and AI-generated inferences, your data integrity is at risk.*

**Executive Summary:**

**In traditional software, a database record represents truth. In AI-native applications, records are a mix of immutable facts and probabilistic AI opinions. Storing an LLM's classification directly on a core entity table inevitably leads to multi-tenant data corruption and prevents historical auditing. This paper advocates for a defensive data modeling paradigm: isolating AI judgments into dedicated 'Evidence Ledgers.'**

## 1. The Engineering Challenge

In version 1 of our discovery platform, AI-derived attributes (e.g., 'job relevance score' or 'extracted skills') were stored as standard columns directly on the main `opportunity` table. When we transitioned to a multi-user environment, a massive conflict arose: User A's AI-curated preference silently overwrote the underlying record for User B. Furthermore, when the LLM prompt was updated, we had no structural way to distinguish legacy inferences from new ones.

In high-stakes enterprise environments, this kind of failure is unacceptable. When building systems for Fortune 500 organizations, relying on basic prompt engineering or out-of-the-box vendor SDKs frequently masks underlying architectural fragility. The goal is to move beyond simple proofs-of-concept into deterministic, resilient data flow.

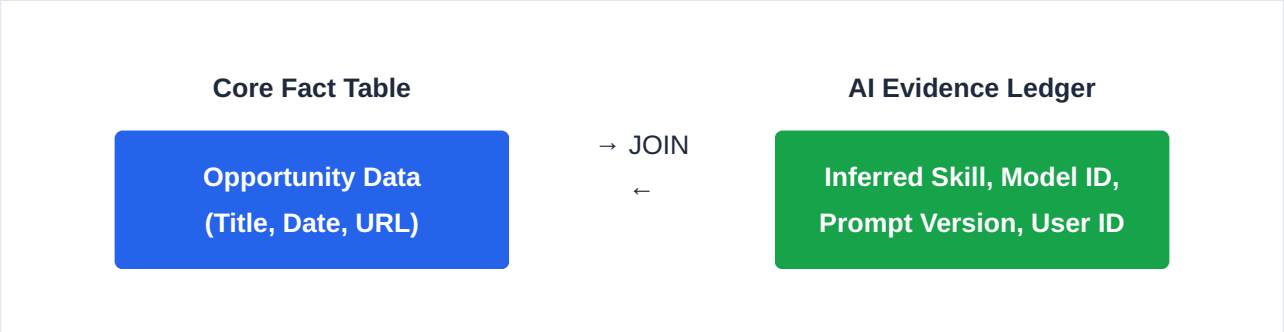
## 2. The Architectural Solution

We transitioned to a strict Fact/Opinion split. The core tables (`job_listing`, `company`) are completely locked down to universal facts. Every AI inference is written to a dedicated `ai_output` ledger table. This table stores the inferred value, the user ID, the specific LLM model used, the prompt version, and the cryptographic hash of the input text. Business

logic queries the facts and JOINs the relevant AI opinions at read-time, ensuring perfect multi-tenant isolation and complete LLM auditability.

By enforcing this pattern at the architectural level, the system no longer relies on the "magic" of generative fluency. Instead, it relies on rigorous data engineering and precise, targeted models. This decoupling ensures that failures are isolated, auditable, and easily corrected without unwinding the entire data pipeline.

## 3. Structural Data Flow




## 4. Business Impact & ROI

For organizations operating at scale, migrating to this pattern fundamentally alters the cost and reliability curve. It shifts the burden of trust from the LLM's inherently probabilistic nature to a deterministic data engineering layer. The result is a system that can process massive throughput securely, drastically reducing API token burn and ensuring complete auditability for internal compliance boards.

---

Published by RKASINC Enterprise Architecture & AI Strategy